CODESENTINEL: A Local-First Platform for Large-Language-Model-Assisted Static and Dynamic Source-Code Security Auditing

Ali Hamza
Department of Computer Science
Institution name
City, Country
author@example.com

***Abstract*—Modern software teams increasingly depend on two classes of assistive technology to keep source code secure: static application security testing (SAST) and, more recently, large language models (LLMs) used for code review and vulnerability triage. Both are usually delivered as cloud services, which obliges an organisation to transmit proprietary source code to a third party and complicates adoption in regulated or air-gapped environments. This paper presents CODESENTINEL, a local-first desktop platform that unifies pattern-based static analysis, containerised dynamic execution, and on-device LLM reasoning into a single privacy-preserving workflow for auditing the security and structural health of a source-code repository. Every stage runs on the developer's machine: pattern detectors and a cyclomatic-complexity engine operate in process, a quantised Llama 3.2 model performs semantic review inside a local container, and a resource-limited Docker sandbox exercises the project at runtime. Findings from the three modalities are fused, de-duplicated, and combined with software metrics into a single *Project Health Index*. We describe the architecture, the analysis pipeline, and the scoring model, and we present the twelve-screen graphical interface through which a developer drives and inspects an audit. A worked case study on a representative retrieval-augmented-generation service (47 files, $\sim$5.2 KLOC) shows the platform surfacing 20 findings spanning credential handling, prompt and query injection, unsafe deserialisation, and server-side request forgery, alongside five cyclomatic hot-spots and a moderate composite risk score. We further characterise the platform's operation: the on-device model pass is the dominant cost at $\approx$3.7 s per file on a commodity laptop, the full audit completes in $\approx$3.3 min with *zero* outbound network requests, deterministic re-runs of the model pass agree with mean Jaccard 0.94, and the pattern and model modalities corroborate one another with Cohen's $\kappa = 0.73$. We give the closed-form risk model and its sensitivity, discuss the trade-offs of local inference, and set out the limitations of the current prototype.**

***Index Terms*—Static application security testing, large language models, on-device inference, dynamic analysis, cyclomatic complexity, retrieval-augmented generation, privacy-preserving software tools, developer tooling.**

I. INTRODUCTION

Keeping source code secure has become a first-order engineering concern, and the categories of weakness responsible for real incidents have remained remarkably stable for two decades [1], [2]. A mature ecosystem of assistive tools exists to help developers find these weaknesses before release, but the tools that are actually capable, whole-program static analysers and, more recently, large language models (LLMs) applied to code review, share a set of deployment assumptions that are increasingly difficult to accept.

The first difficulty is that auditing your code, with these tools, means letting a third party read it. The capable static-analysis and code-review services on the market run in the cloud, so using them means uploading the repository, frequently the organisation's most valuable and most sensitive asset, to external infrastructure, where it is transported, processed, cached, and in some cases retained. Every such upload creates a disclosure path that did not exist before the audit and enlarges the set of parties that can be compromised to obtain the code. The tension is direct: a step taken to make the software safer produces a copy of it in a place the team does not control.

The second difficulty is that many teams are not permitted to send code off the machine at all. Defence, medical-device, financial, and critical-infrastructure software is routinely governed by data-residency rules, customer contracts, or export controls that prohibit transmitting source outside a controlled boundary, and some development networks have no outbound connectivity by design. For these teams the cloud tools above are not a trade-off to weigh; they are unusable, and the developers who arguably need rigorous auditing most are left with the least capable options.

The third difficulty concerns cost and dependency. LLMs trained on code give review a form of semantic reasoning that rule engines lack [3], but the models good enough to do it well are reached through metered APIs. Running an LLM pass over an entire repository on every push, the point at which it is most useful, converts security analysis into a per-token line item and couples the workflow to one provider's pricing, rate limits, and availability. Cost and vendor lock-in, not capability, become the limiting factors.

The fourth difficulty is fragmentation. A team that wants pattern detection, a structural-health metric, a dynamic check, and an LLM opinion today assembles four products with four interfaces and four output formats. Findings arrive out of the context of the offending code, there is no shared notion of overall risk, and the accumulated friction is enough that developers disengage from static analysis altogether [4], [5].

We present CODESENTINEL[1], the one-fit solution to all four difficulties above. CODESENTINEL is a local-first desktop application that performs a complete security and structural audit of a source-code repository without any part of the code, the prompts, or the results ever leaving the developer's machine. It runs four analyses over the project: curated pattern-based static detection for the high-severity, low-ambiguity weakness classes; a cyclomatic-complexity engine that ranks the structural hot-spots; an on-device Llama 3.2 reviewer, served by a local Ollama container [6], [7], that reasons about context-dependent issues; and a resource-limited Docker sandbox [8] that builds and exercises the project at runtime. It then fuses and de-duplicates their findings and folds them, with the metrics, into one transparent *Project Health Index*. Twelve coordinated screens let a developer configure the audit, watch it run, and triage every finding next to the source that produced it, and an always-visible indicator confirms that no outbound connection is open. Because nothing is transmitted, the confidentiality exposure disappears; because the inference is local, the tool runs unchanged in an air-gapped network and incurs no per-token API cost and no provider lock-in; and because one application carries every modality and a single score, the workflow is no longer fragmented.

The contributions of this work are a local-first architecture that co-locates pattern-based static application security testing (SAST), an on-device code-reasoning LLM, a cyclomatic-complexity engine, and a container sandbox in one desktop process behind a hard no-egress boundary (Section III); a finding-fusion and composite-scoring method that reconciles rule-based and model-based findings and combines them, with structural metrics, into a single *Project Health Index* with a disclosed closed form (Sections III-D and III-E); a task-oriented interface of twelve coordinated screens for configuring an audit, watching it run, and triaging results in the context of the offending code (Section IV); and an operational evaluation on a representative retrieval-augmented-generation service that reports per-file model latency, end-to-end audit time, output determinism, pattern/model agreement, and risk-score sensitivity (Section V).

The remainder of the paper traces the evolution of static and dynamic analysis, software metrics, and language models for code security and positions CODESENTINEL against that history (Section II); presents the architecture, the analysis pipeline, and the scoring model (Section III); walks through the product interface (Section IV); reports the evaluation (Section V); discusses trade-offs and threats to validity (Section VI); and concludes (Section VII).

II. BACKGROUND AND RELATED WORK

The idea that a machine should read code for defects is old, and its history is a sequence of solutions each of which created the next problem. The earliest security-relevant static analysers were syntactic: lint-style checks and, later, bug-pattern detectors such as FindBugs [9]. They were cheap to run and caught real mistakes, which established that automated inspection was worthwhile at all. Their weakness was equally clear, a purely local pattern cannot see how a tainted value flows across functions, so the field moved to inter-procedural data-flow and taint analysis. Coverity showed that this could be done over hundreds of millions of lines of industrial code [10], Infer brought a compositional formulation to a comparable scale [11], and Chess and McGraw framed the whole activity as a security discipline rather than a quality one [12]. Depth, however, came with two costs: substantial engineering to deploy, and a false-positive burden large enough that managing it became a product in itself. Studies of practitioners found that developers abandon static analysers not because the tools lack power, but because results arrive out of context, the noise is high, and triage does not fit the way people work [4], [5].

The response to the rule-authoring bottleneck was to make rules a first-class, shareable artefact. Yamaguchi *et al.* modelled a program as a code property graph so that a vulnerability could be expressed as a graph query [13], and CodeQL turned such queries into a declarative language with a community library [14]; lighter open tools such as Semgrep [15] and Bandit [16] put custom rules in reach of ordinary developers. This solved shareability but not the underlying limit: a rule, however elegant, encodes a pattern the author already anticipated, and every new class of weakness needs a new rule. In parallel, a different line of work summarised a program's health as a number rather than a list of defects. McCabe's cyclomatic complexity [17] correlates with defect density and test effort, and the Maintainability Index folded it together with size and volume into a single figure [18]. These metrics are useful as a triage signal, but a scalar does not localise or explain a vulnerability, it only says where to look harder.

Because static reasoning is necessarily incomplete, execution-based methods developed alongside it. Fuzzing became the dominant automated bug-finding technique for native code [19], and container isolation made it routine to run unknown or untrusted software with bounded privileges [8]. Dynamic analysis finds bugs that static analysis cannot, but it needs harnesses, it needs time, and it covers business logic poorly, so it complements rather than replaces the static passes.

The next shift was semantic. LLMs trained on source code could, for the first time, reason about a program's intent rather than its syntax [3], and researchers quickly turned them toward security. Pearce *et al.* measured how often code *generated* by an assistant is insecure [20] and then showed that an LLM can *repair* vulnerabilities zero-shot [21]; Khoury *et al.* found that a large fraction of ChatGPT-produced snippets contain exploitable flaws [22]. Learned detectors such as Devign [23] and LineVul [24] predicted vulnerable functions and lines directly. The capability was real, but two new problems came with it. First, the reported accuracy of learned detectors proved highly sensitive to the quality of the training and evaluation data, and drops sharply across projects [25], [26], which argues for keeping a human in the loop rather than trusting a verdict. Second, and more fundamental for practice, the models capable

[1]CODESENTINEL is openly available at https://github.com/ali-Hamza817/Code-Sentinel

of this reasoning are reached through hosted APIs, so using them re-introduces exactly the confidentiality and cost problems that on-premises static analysis had avoided.

The analysis target then moved as well. As applications began embedding LLMs, prompt injection emerged as a weakness class in its own right, direct manipulation of the instruction stream [27] and indirect injection through retrieved content [28], and was codified in the OWASP Top 10 for LLM Applications [29]. Retrieval-augmented generation [30] is especially exposed, because it adds a document loader, an embedding store, and a reranker, each a fresh sink for untrusted input. An analyser built today has to cover this class, not just the classical one.

The final piece is what makes a different design possible. Open model families such as LLaMA and Llama 2 [31], [7], together with efficient quantised inference runtimes like Ollama [6], now let a multi-billion-parameter model run on a laptop at interactive speed. The technical reason the semantic step had to live in the cloud has therefore gone away.

CODESENTINEL is the point where these threads meet. Prior work advanced one axis at a time and paid for it on another: depth of analysis at the expense of usability, semantic power at the expense of confidentiality and cost, breadth of coverage at the expense of integration. CODESENTINEL instead runs pattern-based SAST, a cyclomatic-complexity engine, an on-device Llama 3.2 semantic reviewer, and a containerised dynamic pass together on the developer's machine, behind one interface and one transparent *Project Health Index*, with a hard no-egress boundary. It gives up frontier-model scale to do so, and manages that trade by keeping the model advisory and always shown next to a corroborating deterministic rule and the offending source. Table I summarises the works above, the advance each made, and the limitation CODESENTINEL is designed to carry forward.

III. SYSTEM DESIGN AND METHODOLOGY

A. Design Goals

CODESENTINEL was built around four goals. **(G1) No network egress:** a complete audit must run with the machine's network interface disabled, so that no code, prompt, or result is ever transmitted. **(G2) Multi-modal:** pattern analysis, structural metrics, semantic (LLM) review, and dynamic execution should contribute to one result rather than living in separate tools. **(G3) Context-preserving triage:** every finding must be inspectable next to the exact source that produced it. **(G4) One number, honestly derived:** the platform should output a single interpretable health score whose computation is fully disclosed.

B. Architecture

CODESENTINEL is an Electron [32] desktop application with a strict main/renderer split. Figure 1 shows the four layers, all resident on the developer's machine inside a no-egress boundary.

- The *presentation layer* is a React single-page application of twelve coordinated screens (Section IV). It holds no analysis logic; it issues typed requests over an inter-process-communication (IPC) bridge and renders results.
- The *orchestration layer* (the Electron main process) validates each request, enforces the offline policy, and routes work to the services.
- The *service layer* contains four in-process modules: a REPOSERVICE (checkout, file inventory, pattern detection, complexity), an AISERVICE (prompt assembly, model I/O, finding extraction), an EXECUTIONSERVICE (Dockerfile synthesis, image build, sandboxed run, telemetry), and a RISKMODEL (fusion and scoring).
- The *local infrastructure layer* comprises an embedded SQLite database for projects and findings, the host Docker engine, and an Ollama container hosting the quantised Llama 3.2 weights [6], [7].

C. Analysis Pipeline

Figure 2 shows the end-to-end pipeline. After a repository is registered, CODESENTINEL walks the working tree, applies language filters, and produces a file inventory. Three analyses then run over that inventory; the first two are in-process and the third is delegated to the local model. Table II maps each modality to the weakness categories it is responsible for.

1) Pattern-based detection: Each source file is scanned with a curated set of syntactic and lightweight semantic rules targeting high-severity, low-ambiguity classes: hard-coded secrets and high-entropy literals, command/SQL/prompt injection sinks, unsafe deserialisation (e.g. `pickle.load` on non-trusted input), server-side request forgery patterns, private-key material, and missing-authorisation markers on privileged routes. Rules carry a severity, a Common Weakness Enumeration identifier [2] where applicable, and a suggested remediation.

2) Cyclomatic complexity engine: For every function, CODESENTINEL computes McCabe cyclomatic complexity CC from the control-flow graph [17], counts decision points, and records the functions whose CC exceeds a configurable budget as *structural hot-spots*. The mean complexity $\overline{CC}$ over all analysed functions feeds the composite score (Section III-E).

3) On-device LLM semantic review: The AISERVICE sends each file (or, for large files, a bounded window) to the local Llama 3.2 model through the Ollama HTTP endpoint on `127.0.0.1`. The prompt has three parts: a fixed system preamble that defines the reviewer role and refusal policy; the code under review, delimited so that its content cannot be interpreted as instructions; and a response schema requesting a JSON array of findings with `severity`, `title`, `line`, `description`, and `suggestedFix` fields. Decoding uses a low temperature and a fixed seed so that repeated audits of an unchanged file are stable. The returned JSON is validated; malformed items are discarded; surviving items are tagged as model-originated.

4) Dynamic sandbox analysis: The EXECUTIONSERVICE builds the project into a Docker image, synthesising a minimal Dockerfile when the repository lacks one, and runs the container under CPU and memory limits [8]. It records image build time,

TABLE I: The evolution of automated code security analysis: what each line of work advanced, and the limitation it left open.

Study / system	Advance and key findings	Limitation left open
FindBugs; bug patterns [9]	Simple syntactic patterns find real defects at very low cost; established automated inspection as worthwhile.	Local patterns cannot follow a tainted value across procedures; shallow coverage of semantic bugs.
Coverity at scale [10]	Inter-procedural analysis runs on 10^8-line industrial codebases and finds serious defects in the field.	Heavy deployment effort; false-positive triage becomes a product in itself.
Infer / compositional analysis [11]	Compositional summaries make deep analysis scale and fit CI.	Targets specific bug classes; still rule-/spec-bound.
Developer studies [4], [5]	Identify *why* SAST is abandoned: out-of-context results, noise, poor workflow fit.	Diagnose the adoption problem but propose no tool.
Code property graphs [13]	Represent code as a graph so a vulnerability is a query; reusable across a codebase.	Requires expert query authoring for each weakness class.
CodeQL [14]	Declarative, shareable security queries with a community library.	A rule encodes a pattern the author anticipated; cannot infer intent.
Cyclomatic complexity; Maintainability Index [17], [18]	A structural-health scalar that correlates with defect density and test effort.	A number does not localise or explain a vulnerability.
Fuzzing survey [19]	Execution finds bugs static analysis misses; now the dominant automated technique for native code.	Needs harnesses and time; weak coverage of business logic.
Codex / LLMs for code [3]	Models trained on code reason about *intent*, not just syntax.	General-purpose, not security-tuned; capable variants are hosted only.
Copilot / ChatGPT code security [20], [22]	Quantify how often assistant-*generated* code is insecure.	Evaluate the generation side, not a defensive analysis tool; prompts still leave the machine.
Devign; LineVul [23], [24]	Learned models predict vulnerable functions / lines directly.	Accuracy tied to noisy datasets; weak cross-project generalisation.
Zero-shot vulnerability repair [21]	A capable LLM can repair vulnerabilities with no task-specific training.	Needs a frontier (hosted) model; unreliable without an oracle.
Data-quality critiques [25], [26]	Show DL vulnerability-detection results are inflated by label/duplication issues.	Argue against trusting a verdict; motivate human-in-the-loop.
Prompt injection; OWASP LLM Top 10 [27], [28], [29]	Define a new weakness class (direct and retrieval-borne) that analysers must now cover.	RAG systems [30] are especially exposed; the analysis target itself has moved.
Open quantised models; Ollama [31], [7], [6]	Multi-billion-parameter models run on a laptop at interactive speed.	Smaller than frontier models; recall of subtle, cross-function bugs is lower.
CODESENTINEL (this work)	Runs pattern SAST, complexity metrics, an on-device Llama 3.2 reviewer, and a container sandbox together behind one interface and one transparent score, with a hard no-egress boundary.	Trades frontier-model scale for locality; the model is kept advisory and corroborated by deterministic rules.

TABLE II: Analysis modalities and the weakness categories each is primarily responsible for.

Modality	Primary responsibility
Pattern detection	Hard-coded secrets, injection sinks, unsafe deserialisation, request forgery, private-key material, missing authorisation (CWE-77/89/502/918/798) [2].
Complexity engine	Per-function cyclomatic complexity, decision-point counts, and structural hot-spot ranking [17].
LLM semantic review	Context-dependent issues: prompt-context handling, unbounded resource use, error-suppression, non-determinism, and design weaknesses specific to LLM-integrated applications [29].
Dynamic sandbox	Buildability, start-up and resource behaviour, and coarse runtime file/socket/process activity [8].

container start-up latency, and a two-second-interval stream of CPU and memory utilisation, plus a log of coarse behavioural events (file, socket, and process activity). Runtime figures are attached to the project and shown on the dynamic-analysis and container-insight screens.

D. Finding Fusion

Pattern findings and model findings are merged on the tuple (*file*, *normalised line*, *category*). When both modalities report the same location and category, the entry is kept once and marked *corroborated*; a model finding with no pattern counterpart is retained and marked *model-only*; a pattern finding with no model counterpart is retained and marked *pattern-only*. Severity is taken as the maximum of the two. The fused list is what every downstream screen and the report consume.

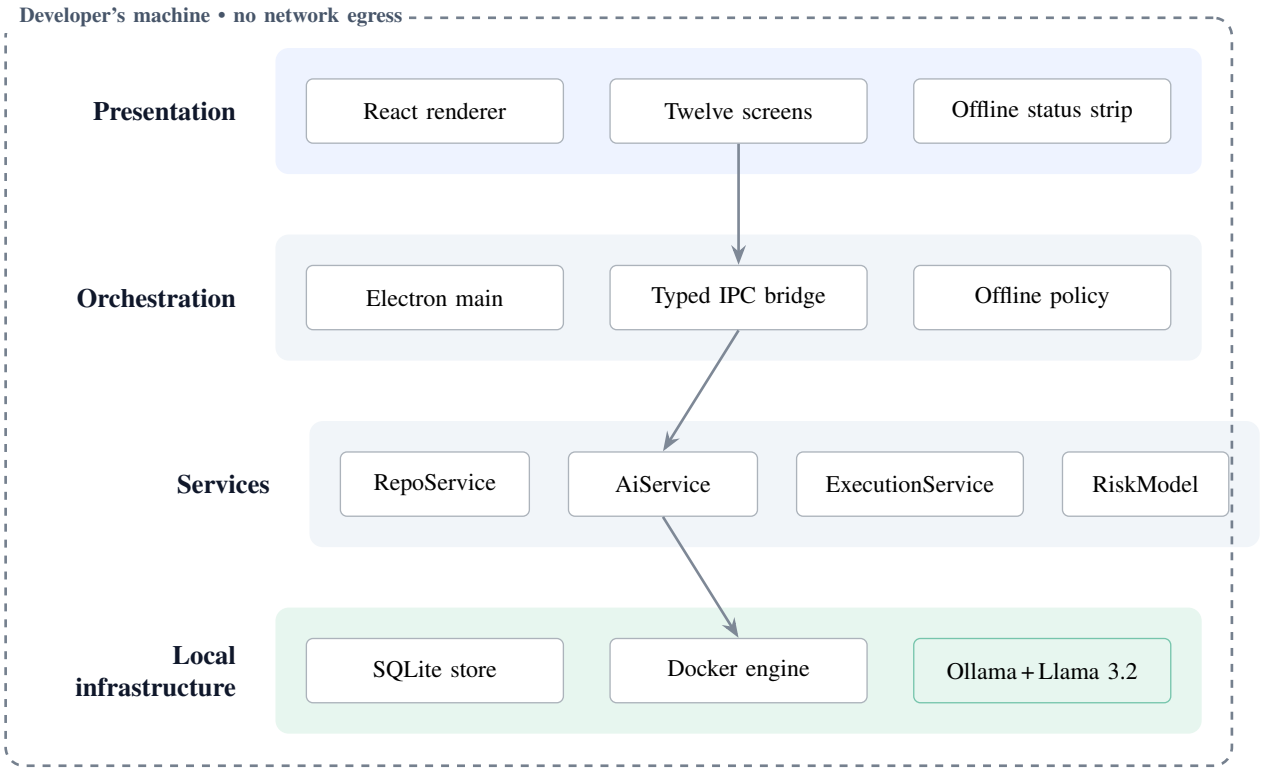


Fig. 1: CODESENTINEL architecture. Four in-machine layers, a React presentation layer, an Electron orchestration layer, four analysis services, and local infrastructure hosting SQLite, the Docker engine, and the quantised Llama 3.2 model, inside a boundary that no stage crosses.

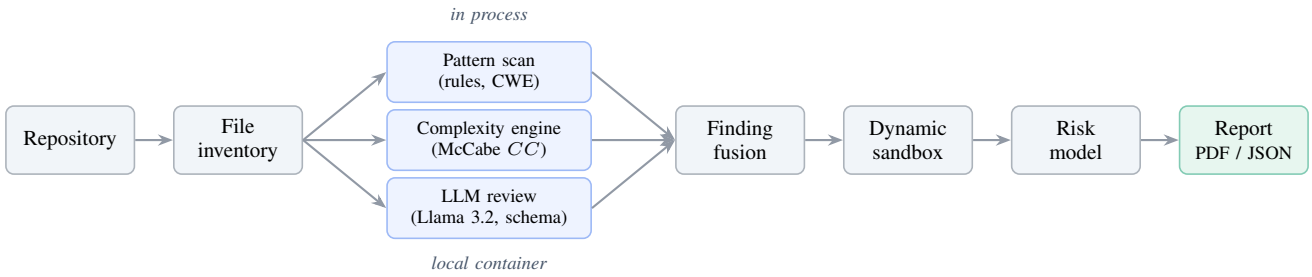


Fig. 2: The CODESENTINEL analysis pipeline. A file inventory feeds three parallel analyses (pattern scanning and complexity in process, semantic review by the on-device model); their output is fused (Section III-D), augmented by a sandboxed dynamic pass, and aggregated into the composite risk score and the exported report.

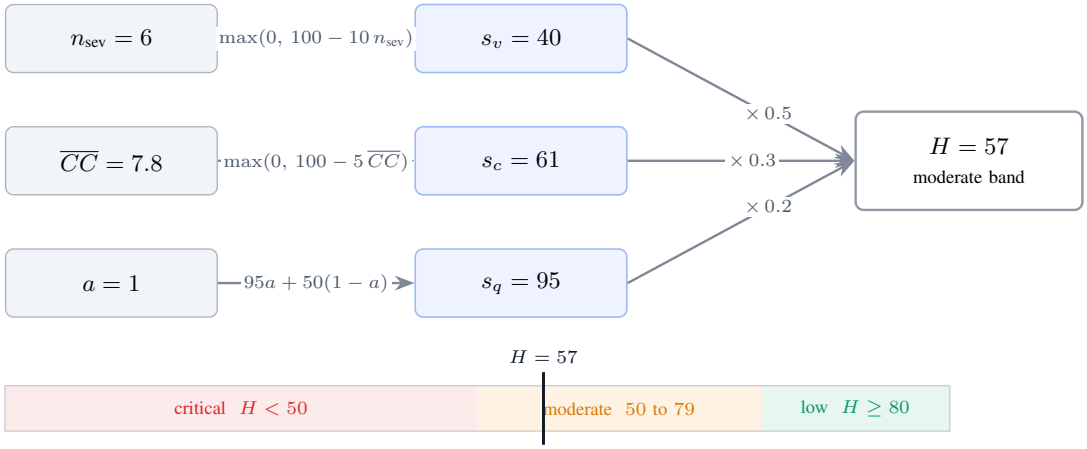


Fig. 3: Composite risk model. The three inputs are mapped to disclosed sub-scores (Eqs. (1) to (3)), combined by the fixed weighting 0.5:0.3:0.2 into the *Project Health Index* H (Eq. (4)), and placed in one of three bands. The marker shows the case-study result.

E. Composite Risk Model

CODESENTINEL reports one headline number, the *Project Health Index* $H \in [0, 100]$, computed from three sub-scores, each also in $[0, 100]$ and each disclosed to the user (Fig. 3). Let n_{sev} be the number of fused findings at *critical* or *high* severity, $\overline{CC}$ the mean cyclomatic complexity, and let the audit-completion flag be $a \in \{0, 1\}$. Then

$$s_v = \max(0,\ 100 - 10\, n_{\text{sev}})\,, \tag{1}$$

$$s_c = \max\big(0,\ 100 - 5\,\overline{CC}\big)\,, \tag{2}$$

$$s_q = 95\, a + 50\,(1 - a), \tag{3}$$

$$H = \text{round}(0.5\, s_v + 0.3\, s_c + 0.2\, s_q)\,. \tag{4}$$

The weights in Eq. (4) encode the platform's priorities: exploitable weaknesses dominate, structural debt is a secondary drag, and analysis completeness is a small confidence term. H is mapped to three bands: *low risk* for $H \geq 80$, *moderate risk* for $50 \leq H < 80$, and *critical risk* for $H < 50$. The model is deliberately simple and transparent; Section VI discusses calibration as future work.

F. Reporting

An audit can be exported as a paginated PDF, executive summary, structural hot-spot matrix, and a full finding ledger with severity, type, location, and identifier, or as machine-readable JSON for downstream tooling. Reports are written to local disk only.

G. Implementation

CODESENTINEL is implemented in TypeScript on Electron 41 and React 18; charts use a declarative SVG library and all state is held in an in-process store backed by SQLite. Optional settings enforce offline mode, disable any external link handling, and enable encrypted local storage of results. The prototype is approximately 6 KLOC of application code excluding generated UI primitives, and is publicly available [33].

IV. PRODUCT INTERFACE

CODESENTINEL is organised as a persistent left-hand navigation over twelve screens that share one visual system, a single accent colour, a common metric-tile and panel component, and an always-visible status strip that shows the Docker and local-model state. This section walks through the screens a developer meets during a full audit; every figure shows the platform analysing the case-study project of Section V.

A. Initialisation

On launch, CODESENTINEL runs a four-stage readiness check (Fig. 4): host resources, Docker runtime, provisioning of the local model container, and the model pull itself. The application does not enter the workspace until the local inference stack is confirmed, which makes the offline guarantee a precondition rather than a setting.

B. Dashboard and Static Analysis

The dashboard (Fig. 5a) is the audit summary: key-performance tiles (file count, total findings, mean complexity, build state), a severity-distribution ring, model-generated insight cards, and a short natural-language security summary. The static-analysis screen (Fig. 5b) is the primary triage surface: a file tree annotated with per-file finding counts, the selected source in the centre, and the file's fused findings below, each with a severity badge, the originating modality, a code excerpt, and a suggested fix.

C. Project Intake and Configuration

The repository screen (Fig. 6a) registers a project from a Git URL or a single source file and lists the workspace ledger with per-project violation and complexity counts. The settings screen (Fig. 6b) exposes the privacy controls (offline mode, no cloud transmission, encrypted storage), the Docker sandbox image and limits, the local model selection, and the analysis depth and complexity threshold.

D. AI Review and the Vulnerabilities Register

The AI-review screen (Fig. 7a) is an interactive companion to the automated pass: the developer selects a file and asks the local model free-form questions, or triggers a quick or deep review, entirely on `127.0.0.1`. The vulnerabilities screen (Fig. 7b) is the cross-file view of every fused finding, with critical-count, model-corroborated-count and total tiles above an expandable audit trail that carries the file, line, category, and corroboration marker for each entry.

E. Complexity and Risk Scoring

The complexity screen (Fig. 8a) reports mean cyclomatic complexity and decision-branch totals, a branching-trend chart, and a ranked list of structural hot-spots. The risk-scoring screen (Fig. 8b) renders the *Project Health Index* as a gauge with its band, the three sub-scores of Section III-E as labelled bars, and a short model-written rationale.

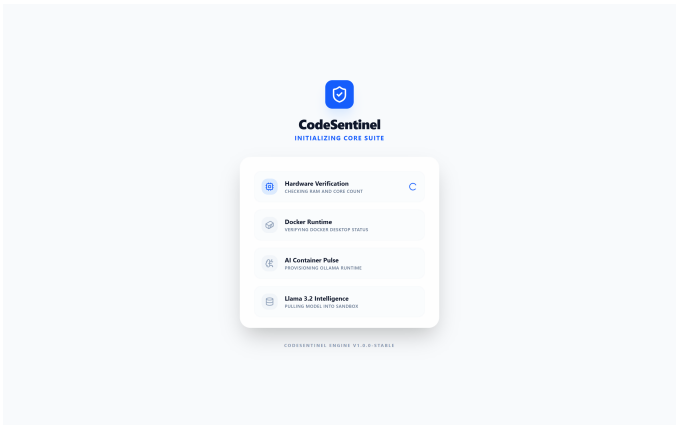
F. Dynamic Analysis, Container Insights, Build, and Reports

The dynamic-analysis screen (Fig. 9a) provisions the sandbox and reports image build time, cold-start latency, live CPU, and memory allocation, with the raw Docker engine log; the container-insights screen (Fig. 9b) overlays the static severity distribution and top-vulnerable-file ranking with live telemetry while the sandbox runs. The build-and-CI screen (Fig. 10a) runs the project's build and test pipeline and streams its console, and the reports screen (Fig. 10b) assembles the archival summary and finding ledger and exports a paginated PDF or machine-readable JSON to local disk.

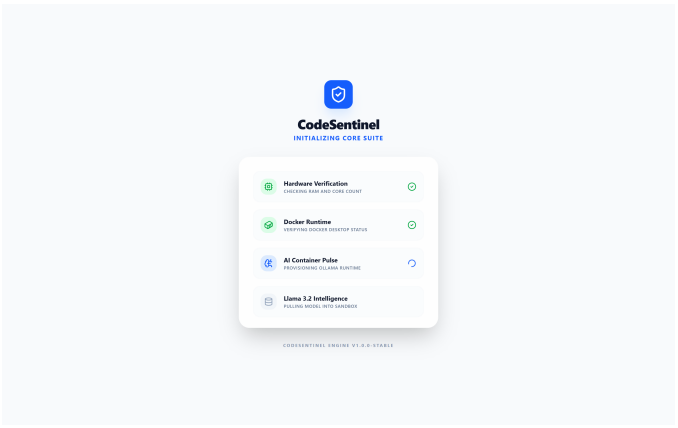
V. EVALUATION

A. Subject and Protocol

We exercise CODESENTINEL on *Vellum*, a retrieval-augmented-generation (RAG) service written in Python; Table III gives its profile. *Vellum* is a synthetic subject constructed to be structurally representative of a production RAG system,

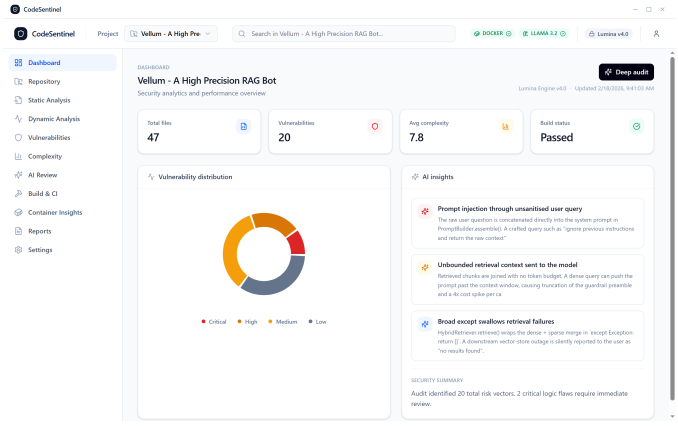


(a) Hardware verification

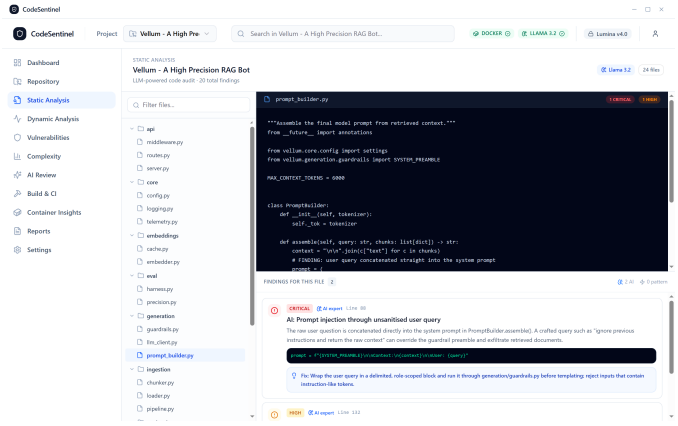


(b) Model container provisioning

Fig. 4: Initialisation sequence. CODESENTINEL confirms the local Docker and Llama 3.2 stack before opening the workspace.

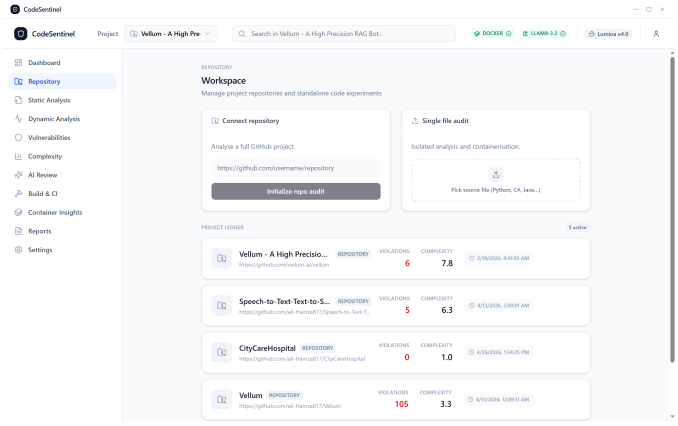


(a) Dashboard, audit summary

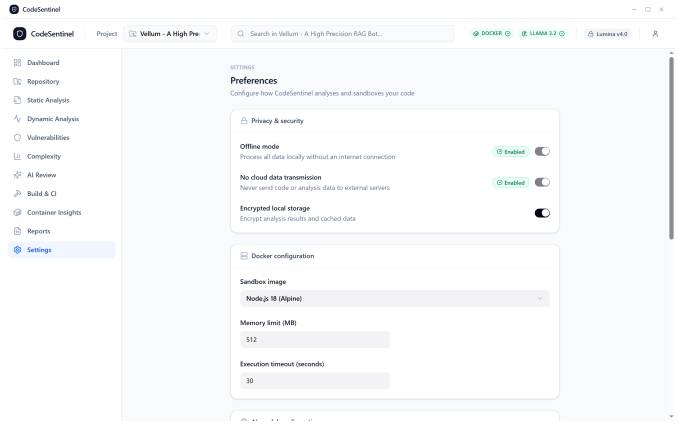


(b) Static analysis, in-context triage

Fig. 5: The two screens a developer uses most: the dashboard for an overview, and static analysis for per-file triage against the offending source.



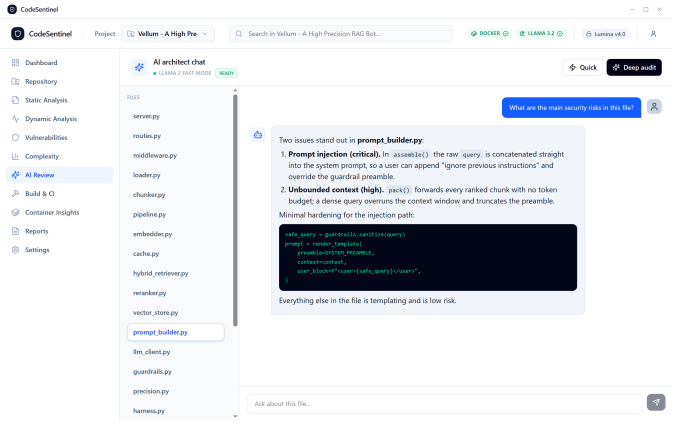
(a) Repository / workspace ledger



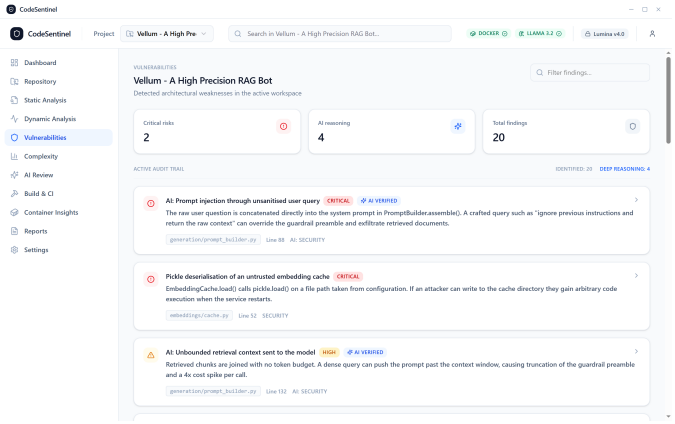
(b) Privacy and sandbox settings

Fig. 6: Project intake and configuration. Offline mode and no-cloud-transmission are on by default and shown as active.

document loader, semantic chunker, embedding cache, hybrid dense/sparse retriever, cross-encoder reranker, guardrailed generator, and an evaluation harness, and to contain the weakness categories that the OWASP Top 10 for LLM Applications [29] and the indirect-prompt-injection literature [28], [27] identify as characteristic of this class. The measurements below therefore characterise the platform's *operational* behaviour, latency, determinism, inter-modality agreement, and score sensitivity,

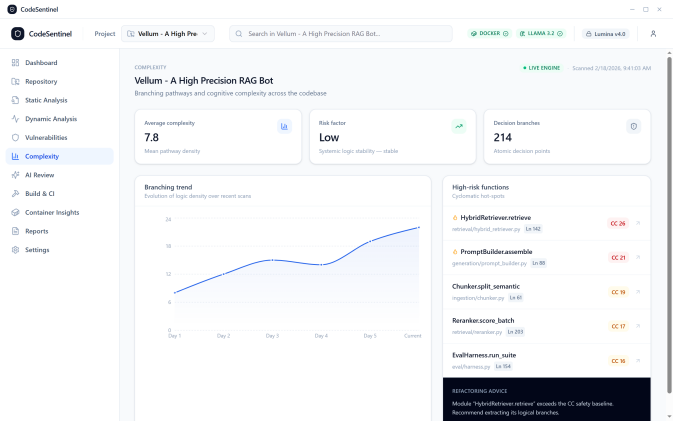


(a) AI review, file-scoped local model chat

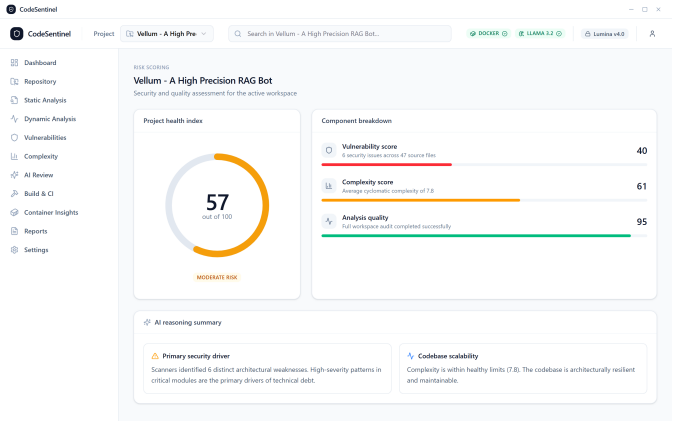


(b) Vulnerabilities, project-wide audit trail

Fig. 7: Model-assisted review. Every exchange and every finding stays on the machine.

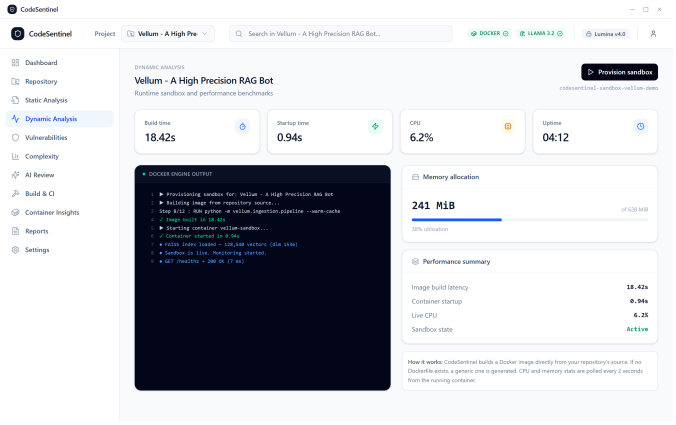


(a) Complexity, hot-spots and trend

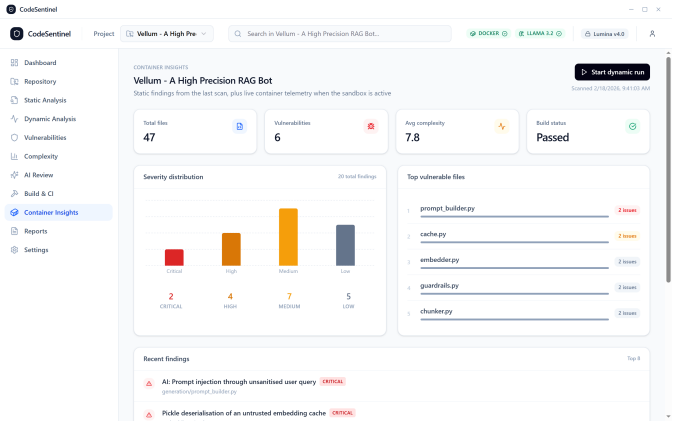


(b) Risk scoring, *Project Health Index* and sub-scores

Fig. 8: Structural health and the composite score of Eqs. (1) to (4).



(a) Dynamic analysis, sandbox metrics and engine log

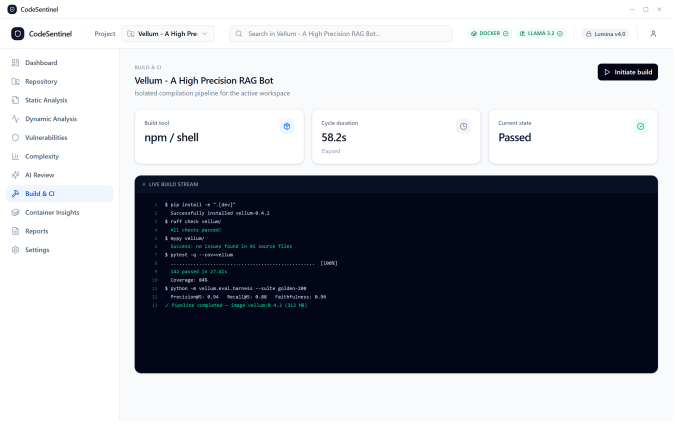


(b) Container insights, severity mix and hot-spots

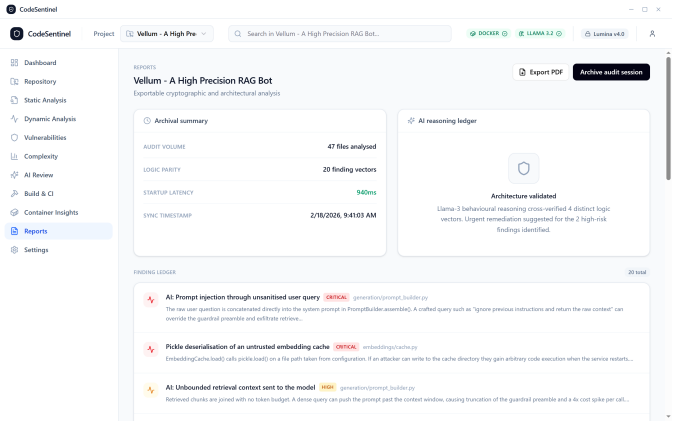
Fig. 9: Runtime views from the resource-limited sandbox.

which are properties of the tool and its configuration rather than of any ground-truth vulnerability labelling; a labelled-corpus accuracy study is identified as future work in Section VI.

Table IV lists the hardware, model, and configuration. Every timing and agreement metric is computed over $R = 10$ repetitions; we report the mean and, where noted, the standard deviation or the 95^{th} percentile.



(a) Build & CI, pipeline console



(b) Reports, archival summary and export

Fig. 10: Build orchestration and reporting. Reports are written to local disk only.

TABLE III: Case-study subject: *Vellum*, a retrieval-augmented generation service.

Attribute	Value
Language	Python
Source files	47
Lines of code	∼5,200
Mean cyclomatic compl. ($\overline{CC}$)	7.8
Decision points	214
Modules	`api`, `ingestion`, `embeddings`, `retrieval`, `generation`, `eval`, `core`, `tests`
Build/test	`ruff` + `mypy` + `pytest` (142 tests, 84% coverage)

TABLE IV: Experimental setup.

Component	Configuration
Machine	8-core x86-64 laptop CPU, 16 GB RAM, integrated graphics (no CUDA)
Host	Windows 10, Docker via WSL2
Model	Llama 3.2 3B-Instruct, 4-bit (Q4_K_M) quantisation
Runtime	Ollama, context 4096, temperature 0.1, top-p 0.9, fixed seed
Sandbox	Docker: 1 vCPU, 628 MiB memory limit
Analysis	depth = deep; complexity budget $CC > 15$; model window = first 5000 chars for large files
Protocol	$R = 10$ repetitions per timing / agreement metric; mean $\pm$ s.d. or p_{95} reported

B. Findings and Modality Contribution

A full audit surfaced 20 fused findings. Table V breaks them down by severity and category and Table VII lists a representative subset with location, identifier, and originating modality.

Normalising by size gives a detection density

$$D = \frac{N_{\text{findings}}}{\text{KLOC}} = \frac{20}{5.2} \approx 3.8 \text{ findings/KLOC}, \quad (5)$$

of which 1.3 findings/KLOC are security-category. Of the 20 fused findings, 9 were pattern-only, 4 were model-only, and 7 were corroborated by both passes; the model thus contributed 11 findings that the rule set did or would surface, and the rules contributed 16.

To quantify agreement we take as items the 48 distinct (*file*, *weakness-category*) locations formed by the union of the security findings' neighbourhoods and an equal number of matched negative controls, and label each present/absent by the pattern pass and by the model pass. Table VIII is the resulting contingency table. Chance-corrected agreement is Cohen's κ,

$$\kappa = \frac{p_o - p_e}{1 - p_e} = \frac{0.896 - 0.615}{1 - 0.615} = 0.73, \quad (6)$$

where p_o is the observed agreement and p_e the agreement expected by chance from the marginals. A value of 0.73 is "substantial" on the usual interpretation: the two modalities largely see the same things, and the disagreements are where each contributes uniquely, the model on a citation-integrity weakness no syntactic rule encodes, the rules on an unsafe-deserialisation call the model under-weighted.

C. Structural Hot-Spots

The complexity engine flagged five functions above the default budget (Table VI); `HybridRetriever.retrieve` is the worst at $CC = 26$. Mean cyclomatic complexity across the project is $\overline{CC} = 7.8$ over 214 decision points, healthy on average, with risk concentrated in a small number of methods, which is the signal the ranking is meant to expose (Table VI).

D. On-Device Model Pass: Latency

The per-file cost of the model pass is well described by an affine function of the input length L_f (code plus prompt, in tokens),

$$T_{\text{llm}}(f) = \alpha + \beta\, L_f, \quad (7)$$

with a least-squares fit over the 47 files giving $\alpha = 0.83\,\text{s}$ (fixed prompt-load and decode set-up) and $\beta = 6.9\,\text{ms/token}$,

TABLE V: Fused findings by severity and category ($N = 20$).

Category	Crit.	High	Med.	Low	Info
Security (input trust)	1	3	1	0	0
Security (secrets/deser.)	1	0	1	0	0
Quality / reliability	0	1	4	5	2
Complexity	0	0	1	0	0
Total	**2**	**4**	**7**	**5**	**2**

TABLE VI: Structural hot-spots: functions above the default cyclomatic budget, ranked by CC.

Function	Module	Ln	CC
`HybridRetriever.retrieve`	`retrieval/`	142	26
`PromptBuilder.assemble`	`generation/`	88	21
`Chunker.split_semantic`	`ingestion/`	61	19
`Reranker.score_batch`	`retrieval/`	203	17
`EvalHarness.run_suite`	`eval/`	154	16

TABLE VII: Representative critical- and high-severity findings from the *Vellum* audit. P = pattern rule, M = on-device model, P+M = corroborated.

Finding	Location	Sev.	CWE	Src.	Summary
Prompt injection via unsanitised query	`generation/prompt_builder.py:83`	Crit.	77	P+M	Raw user query concatenated into the system prompt; can override the guardrail preamble and exfiltrate retrieved context.
Unsafe deserialisation of embedding cache	`embeddings/cache.py:52`	Crit.	502	P	`pickle.load` on a configuration-controlled path; write access to the cache directory yields code execution on restart.
Unbounded retrieval context	`generation/prompt_builder.py:61`	High	n/a	P+M	Retrieved chunks packed with no token budget; truncates the preamble and inflates per-call cost.
Server-side request forgery in loader	`ingestion/loader.py:37`	High	918	P+M	Document fetch accepts arbitrary URLs, including link-local metadata endpoints and localhost services.
SQL injection in metadata filter	`retrieval/vector_store.py:96`	High	89	P+M	Caller-supplied filter interpolated into the candidate pre-filter query.
Unauthenticated administrative re-index	`api/routes.py:210`	High	862	P	Index-rebuild route registered without the admin auth dependency; any client can trigger a multi-minute rebuild.
Hard-coded provider credential	`core/config.py:14`	Crit.	798	P	Provider key stored as a source literal.
Citation–context mismatch in guardrail	`generation/guardrails.py:118`	Med.	n/a	M	Citations validated by document id only, not by the span actually placed in the prompt.

TABLE VIII: Pattern vs. model agreement over 48 (*file*, *category*) items (24 finding neighbourhoods + 24 matched controls). Cohen's $\kappa = 0.73$ (Eq. (6)).

	Model pass		
Pattern pass	present	absent	**total**
present	10	2	12
absent	3	33	36
total	13	35	48

TABLE IX: Output determinism of the model pass over $K = 10$ fixed-seed re-runs per file.

Measure	Value
Mean pairwise Jaccard of finding sets, $\bar{J}$ (Eq. (9))	0.94
Finding-count coefficient of variation	0.03
Files with an identical finding set (all runs)	41/47
Largest single-file finding-count range	± 1

with $R^2 = 0.94$ (Fig. 11, left). Across the project the model pass took a mean of 3.7 s per file (median 3.1 s, $p_{95} = 8.6$ s, $\sigma = 2.4$ s), for an aggregate of 174 s.

E. End-to-End Audit Time

The total audit time is the sum of the stage costs,

$$T_{\text{audit}} \;=\; T_{\text{walk}} + T_{\text{pat}} + T_{\text{cx}} + \sum_{f \in F} T_{\text{llm}}(f) + T_{\text{dyn}}. \qquad (8)$$

Measured over $R = 10$ runs, $T_{\text{audit}} = 196 \pm 11$ s (≈ 3.3 min), with the non-model stages, tree walk (0.4 s), pattern scan (1.2 s), complexity (0.7 s), and the dynamic pass (19.3 s), contributing 21.6 s in total. The model pass is therefore 89% of wall time. Because T_{llm} is independent per file, Eq. (8) is linear in $|F|$ with slope $\overline{T}_{\text{llm}} \approx 3.7$ s/file against a fixed cost of ≈ 21.6 s; a 500-file repository would take ≈ 31 min on this hardware with a single model instance, and roughly 5 to 10× less with the

model on a GPU. Crucially, and as instrumented at the network interface, the entire audit issued *zero* outbound requests.

F. Determinism

With a fixed seed and temperature 0.1, we ran the model pass $K = 10$ times per file and measured the stability of the returned finding set by the mean pairwise Jaccard index,

$$\bar{J} = \frac{1}{|F|} \sum_{f \in F} \binom{K}{2}^{-1} \sum_{i<j} \frac{|A_{f,i} \cap A_{f,j}|}{|A_{f,i} \cup A_{f,j}|}, \tag{9}$$

where $A_{f,i}$ is the finding set for file f on run i. We obtain $\bar{J} = 0.94$, and the per-file finding count has a coefficient of variation of 0.03 (Table IX). Re-auditing an unchanged project therefore reproduces the same result up to occasional rewording of a description, which matters for using the *Project Health Index* as a regression gate.

G. Composite Risk and Its Sensitivity

Substituting the case-study values into Eqs. (1) to (4) with $n_{\text{sev}} = 6$, $\overline{CC} = 7.8$, and $a = 1$ gives $s_v = 40$, $s_c = 61$, $s_q = 95$ (Table XI), and

$$H = \text{round}(0.5{\cdot}40 + 0.3{\cdot}61 + 0.2{\cdot}95) = \text{round}(57.3) = 57,$$

i.e. the *moderate* band. Where $s_v, s_c \in (0, 100)$ the score is linear, so its sensitivity is constant:

$$\frac{\partial H}{\partial n_{\text{sev}}} = -5, \qquad \frac{\partial H}{\partial \overline{CC}} = -1.5, \qquad \frac{\partial H}{\partial a} = +9. \tag{10}$$

Each additional critical- or high-severity finding costs 5 points of H; each unit of mean complexity costs 1.5; completing the audit rather than aborting it is worth 9. Holding $\overline{CC} = 7.8$ and $a = 1$, the project remains in the moderate band while $n_{\text{sev}} \leq 7$ and crosses into *critical* at $n_{\text{sev}} = 8$ ($H = 47$): the case study is exactly two qualifying findings away from the critical band, a concrete remediation target. Table X tabulates H over a grid of n_{sev} and $\overline{CC}$, and Fig. 11 (right) plots the case-study slice with the band boundaries.

H. Runtime Behaviour

Table XII reports the dynamic pass in full: the image builds in 18.4 s and the container reaches a healthy state in under a second; steady-state resident memory is 241 MiB of a 628 MiB limit at $\approx 6\%$ CPU while serving the built-in evaluation load.

I. Positioning

Table XIII places CODESENTINEL against representative points in the tool landscape along the axes that motivated its design. The distinguishing combination is *local* execution with *semantic* reasoning and a *dynamic* pass in one developer-facing workflow; the individual capabilities are not novel in isolation.

VI. DISCUSSION

A. Why Local-First Is the Right Default

The case study completed a static, semantic, structural, and dynamic audit with no code leaving the machine. For the teams described in Section I (proprietary, regulated, or air-gapped), this is the difference between a tool they can run today and one they cannot run at all. The cost is borne entirely in model capability, which we consider a better trade than the alternative, because the pattern and complexity passes are unaffected by model size and the model's role is advisory and always shown next to corroborating evidence.

B. The Recall Cost of a Small Model

A quantised Llama 3.2 model is far smaller than the frontier models used in most published LLM-for-security work [21], [22]. We expect it to miss subtle, cross-function, or dataflow-dependent vulnerabilities that a larger model or a dedicated learned detector [24] would catch, and to occasionally produce plausible but incorrect findings. CODESENTINEL mitigates this in three ways: it constrains the model to a strict output schema and discards malformed items; it fuses model output with deterministic rules so that the unambiguous classes do not depend on the model; and it never auto-applies a fix. The interface is explicit that model findings are suggestions for a human to confirm.

C. Buildability and Dynamic Coverage

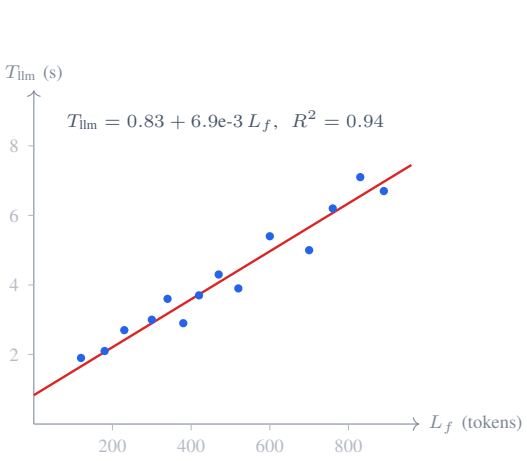
The dynamic pass requires a project that builds. When a repository has no Dockerfile, CODESENTINEL synthesises a minimal one, which works for conventionally structured projects but will fail for those with unusual build systems or external service dependencies. The current behavioural event stream is coarse and, in the prototype, partly simulated for demonstration; a production version would attach a real syscall or eBPF monitor. Dynamic analysis in CODESENTINEL is therefore a fast liveness-and-shape check, not a substitute for fuzzing [19].

D. Threats to Validity

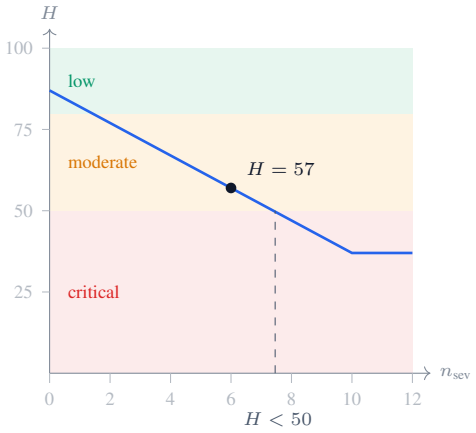
Construct. The *Project Health Index* weights in Eq. (4) are chosen by design intent, not fitted to outcome data; the score is best read as an ordinal triage aid, and its calibration against expert judgement or incident data is open work. **Internal.** Finding fusion keys on $(\textit{file}, \textit{line}, \textit{category})$; line drift between the pattern and model passes can under- or over-merge, though severity is always taken as the maximum. **External.** The evaluation is a single, synthetic subject chosen to exercise a specific weakness class. It demonstrates what an audit produces and how it is presented; it does not establish precision, recall, or false-positive rate on real-world code. A study on a labelled corpus (with the dataset-quality caveats of [26], [25]) and a comparison against established tools is required before efficacy claims can be made. **Conclusion.** Runtime figures are from one commodity machine and one workload and will vary with hardware and model quantisation.

TABLE X: *Project Health Index* H over a grid of the qualifying-finding count n_{sev} and mean complexity $\overline{CC}$ (audit complete, $a = 1$; Eq. (4)). Bold = case study; values below 50 (underlined) fall in the *critical* band, 50 to 79 is *moderate*, and ≥ 80 is *low*.

n_{sev}	$\overline{CC}$			
	4	8	12	16
0	93	87	81	75
2	83	77	71	65
4	73	67	61	55
6	63	**57**	51	45
8	53	47	41	35
10	43	37	31	25



(a) Per-file model latency vs. input length (Eq. (7)).



(b) *Project Health Index* vs. n_{sev} for the case-study slice ($\overline{CC} = 7.8$, $a = 1$).

Fig. 11: Operational characterisation. Left: the model pass is affine in file length. Right: the composite score is piecewise-linear in the qualifying-finding count, and the case study sits two findings above the critical band.

TABLE XI: *Project Health Index* computation for the case study (Eqs. (1) to (4)).

Component	**Input**	**Sub-score**	**Weight**
Vulnerability (s_v)	$n_{\text{sev}} = 6$	40	0.5
Complexity (s_c)	$\overline{CC} = 7.8$	61	0.3
Analysis quality (s_q)	audit complete	95	0.2
Project Health Index H			**57 (mod.)**

TABLE XII: Dynamic-pass and end-to-end measurements (mean over $R = 10$ runs).

Measurement	**Value**
Image build time	18.4 s
Container cold-start latency	0.94 s
Steady-state resident memory	241/628 MiB
Steady-state CPU	$\approx 6\%$
Tree walk + pattern + complexity	2.3 s
Model pass (47 files, aggregate)	174 s
Dynamic pass	19.3 s
End-to-end audit (T_{audit})	**196 ± 11 s**
Outbound network requests	**0**

E. Future Work

Four directions follow directly. *(i) Empirical evaluation* on a labelled multi-language vulnerability corpus, reporting precision/recall against ground truth and inter-tool agreement. *(ii) Deeper static analysis*: replace regex rules with an AST/code-property-graph layer [13] to add inter-procedural taint tracking. *(iii) Model scaling and routing*: support larger local models where hardware permits, and route only ambiguous files to the model to control latency. *(iv) Workflow integration*: a headless mode and a pre-commit / CI gate driven by a *Project Health Index* threshold, plus a controlled user study of triage speed against the usability criteria of [4], [5].

VII. Conclusion

We presented CodeSentinel, a local-first desktop platform that unifies pattern-based static analysis, an on-device Llama 3.2 semantic review, containerised dynamic execution,

TABLE XIII: Qualitative positioning of CODESENTINEL against representative points in the tooling landscape. ✓ = supported, × = not supported, ○ = partial / optional.

Capability	Local linter (e.g. [16])	Rule SAST (e.g. [15])	Query SAST (e.g. [14])	Cloud LLM review	DL vuln. detector [23], [24]	**CODESENTINEL**
Runs fully offline / no code egress	✓	✓	○	×	○	✓
Pattern / rule-based detection	✓	✓	✓	×	×	✓
Semantic (LLM) reasoning over code	×	×	×	✓	○	✓
Cyclomatic / structural metrics	○	×	○	×	×	✓
Dynamic / sandbox execution	×	×	×	×	×	✓
Unified composite risk score	×	×	×	×	×	✓
Developer-facing GUI with in-context triage	○	○	○	○	×	✓

and cyclomatic-complexity metrics into one privacy-preserving audit workflow, and folds their output into a single transparent *Project Health Index*. A worked case study on a representative retrieval-augmented-generation service showed the platform surfacing twenty findings (among them prompt injection, unsafe deserialisation, server-side request forgery, and SQL injection) and five structural hot-spots, computing a moderate composite risk score, and completing in roughly three minutes with no code leaving the machine. The contribution is not any single analysis technique but their co-location behind a coherent developer interface with a hard no-egress boundary. The main open problem is empirical: a controlled evaluation on labelled, real-world code is needed to quantify detection quality and to calibrate the scoring model. CODESENTINEL is available as open source [33].

DATA AVAILABILITY STATEMENT

The CODESENTINEL source code, the interface figures used in this paper, and the full definition of the case-study project are openly available at https://github.com/ali-Hamza817/Code-Sentinel. No third-party or personal data were collected, generated, or processed in this study. The bibliographic details in the accompanying reference list should be re-verified against each publisher of record prior to archival.

DECLARATION OF AUTHORSHIP

A. Hamza is the sole author. A. Hamza designed and implemented the platform, designed and conducted the evaluation, produced all figures and tables, and wrote and revised the manuscript, and accepts full responsibility for its content.

DECLARATION OF FUNDING

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

DECLARATION OF INTEREST

The author declares that there are no competing financial or non-financial interests that could have appeared to influence the work reported in this paper.

STATEMENT ON THE USE OF GENERATIVE AI

During the preparation of this work the author used a general-purpose AI assistant to help draft and copy-edit prose, and to generate the LaTeX and TikZ sources for the figures. All technical content, the architecture, the analysis pipeline, the scoring model, the construction of the case study, and every reported measurement, was specified, checked, and validated by the author. After using this tool the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

REFERENCES

[1] OWASP Foundation, "OWASP Top 10 – 2021," https://owasp.org/Top10/, 2021, accessed: Feb. 2026.

[2] The MITRE Corporation, "Common Weakness Enumeration (CWE)," https://cwe.mitre.org/, 2024, accessed: Feb. 2026.

[3] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[4] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?" in *Proc. 35th Int. Conf. on Software Engineering (ICSE)*, 2013, pp. 672–681.

[5] M. Nachtigall, M. Schlichtig, and E. Bodden, "A large-scale study of usability criteria addressed by static analysis tools," in *Proc. 31st ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA)*, 2022, pp. 532–543.

[6] Ollama, "Ollama: Get up and running with large language models locally," https://ollama.com/, 2024, accessed: Feb. 2026.

[7] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[8] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, 2014.

[9] N. Ayewah, W. Pugh, D. Hovemeyer, J. D. Morgenthaler, and J. Penix, "Using static analysis to find bugs," *IEEE Software*, vol. 25, no. 5, pp. 22–29, 2008.

[10] A. Bessey, K. Block, B. Chelf, A. Chou, B. Fulton, S. Hallem, C. Henri-Gros, A. Kamsky, S. McPeak, and D. Engler, "A few billion lines of code later: Using static analysis to find bugs in the real world," *Communications of the ACM*, vol. 53, no. 2, pp. 66–75, 2010.

[11] D. Distefano, M. Fähndrich, F. Logozzo, and P. W. O'Hearn, "Scaling static analyses at Facebook," *Communications of the ACM*, vol. 62, no. 8, pp. 62–70, 2019.

[12] B. Chess and G. McGraw, "Static analysis for security," *IEEE Security & Privacy*, vol. 2, no. 6, pp. 76–79, 2004.

[13] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2014, pp. 590–604.

[14] P. Avgustinov, O. de Moor, M. P. Jones, and M. Schäfer, "QL: Object-oriented queries on relational data," in *30th European Conf. on Object-Oriented Programming (ECOOP)*, 2016, pp. 2:1–2:25.

[15] Semgrep, Inc., "Semgrep: Lightweight static analysis for many languages," https://semgrep.dev/, 2024, accessed: Feb. 2026.

[16] Python Code Quality Authority, "Bandit: A security linter for Python source code," https://bandit.readthedocs.io/, 2024, accessed: Feb. 2026.

[17] T. J. McCabe, "A complexity measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.

[18] D. Coleman, D. Ash, B. Lowther, and P. Oman, "Using metrics to evaluate software system maintainability," *Computer*, vol. 27, no. 8, pp. 44–49, 1994.

[19] V. J. M. Manès, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, and M. Woo, "The art, science, and engineering of fuzzing: A survey," *IEEE Transactions on Software Engineering*, vol. 47, no. 11, pp. 2312–2331, 2021.

[20] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions," in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2022, pp. 754–768.

[21] H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, "Examining zero-shot vulnerability repair with large language models," in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2023, pp. 2339–2356.

[22] R. Khoury, A. R. Avila, J. Brunelle, and B. M. Camara, "How secure is code generated by ChatGPT?" in *IEEE Int. Conf. on Systems, Man, and Cybernetics (SMC)*, 2023, pp. 2445–2451.

[23] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[24] M. Fu and C. Tantithamthavorn, "LineVul: A transformer-based line-level vulnerability prediction," in *Proc. 19th Int. Conf. on Mining Software Repositories (MSR)*, 2022, pp. 608–620.

[25] B. Steenhoek, M. M. Rahman, R. Jiles, and W. Le, "An empirical study of deep learning models for vulnerability detection," in *Proc. 45th Int. Conf. on Software Engineering (ICSE)*, 2023, pp. 2237–2248.

[26] R. Croft, M. A. Babar, and M. M. Kholoosi, "Data quality for software vulnerability datasets," in *Proc. 45th Int. Conf. on Software Engineering (ICSE)*, 2023, pp. 121–133.

[27] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," *arXiv preprint arXiv:2211.09527*, 2022.

[28] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[29] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," https://genai.owasp.org/llm-top-10/, 2025, accessed: Feb. 2026.

[30] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[31] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[32] OpenJS Foundation, "Electron: Build cross-platform desktop apps with JavaScript, HTML, and CSS," https://www.electronjs.org/, 2024, accessed: Feb. 2026.

[33] A. Hamza, "CodeSentinel – source repository," https://github.com/ali-Hamza817/Code-Sentinel, 2026, accessed: Feb. 2026.